Introducción a las Redes Neuronales

Apuntes — Minería de Datos III

Basado en el material de Ander González Docasal (Mondragon Unibertsitatea)

Índice

1. Redes Neuronales **1**
- 1.1 ¿Qué son las Redes *Neuronales* (NN)? . . . 1
- 1.2 Aprendizaje profundo (*Deep Learning*) . . . 2
- 1.3 Aplicaciones del Deep Learning . . . 2
- 1.4 Grandes Modelos de Lenguaje (LLMs) . . . 2
- 1.5 "Todo es álgebra" . . . 3

1.1 El Perceptrón **4**
- Unidad mínima de cómputo de una NN: neurona o perceptrón . . . 4
- Componentes: vector de pesos y sesgo . . . 4
- Función de activación . . . 4

1.2 Redes Neuronales **6**
- Unión de capas de neuronas . . . 6
- Aprendizaje profundo: capas . . . 6
- Capas ocultas . . . 6
- Formalización completa de una red neuronal . . . 6

1.3 Tipos de salidas **8**
- Tipos de problemas . . . 8
- Problemas de regresión . . . 8
- Problemas de clasificación . . . 8
- Salida de la red . . . 8
- La función Softmax . . . 9
- Función de activación según el tipo de problema . . . 9
- Función de activación en las capas ocultas . . . 10

Resumen final **11**

1. Redes Neuronales

1.1 ¿Qué son las Redes *Neuronales* (NN)?

Las Redes Neuronales (del inglés *Neural Networks,* NN) son modelos matemáticos inspirados —de forma muy laxa— en el funcionamiento del cerebro biológico. El

nombre proviene de su unidad básica de cómputo, la **neurona artificial**, que imita de forma simplificada a la neurona biológica.

1.2 Aprendizaje profundo (*Deep Learning*)

El término **Deep Learning** (aprendizaje profundo) hace referencia al uso de redes neuronales compuestas por múltiples **capas** (*layers*) apiladas unas sobre otras, de forma similar a como una lasaña se compone de múltiples capas superpuestas. Cuantas más capas tiene una red, más "profunda" se considera.

Un ejemplo célebre de arquitectura profunda es la **arquitectura GPT**, basada en bloques *Transformer* apilados, que incluye entre otros los siguientes componentes por bloque:

- *Embedding* de entrada (*Input Embedding*) + codificación posicional (*Positional Encoding*)
- Múltiples cabezas de atención (*Multi-Head Attention*): operaciones de multiplicación matricial (*Matmul*), enmascarado (*Mask*), *Softmax* y *Dropout*
- Capas lineales (*Linear*)
- Normalización de capa (*LayerNorm*)
- Función de activación GELU
- Una capa final con *Softmax* para producir la salida

1.3 Aplicaciones del Deep Learning

Visión por computador (*Computer Vision*):

- Generación de imágenes/vídeo
- Clasificación de imágenes/vídeo
- Detección de posición (localización de objetos)
- Diagnóstico médico/sanitario
- Conducción autónoma

Procesamiento del Lenguaje Natural (NLP, *Natural Language Processing*):

- Comprensión del lenguaje natural (*Natural Language Understanding*)
- Generación del lenguaje natural (*Natural Language Generation*)
- Procesamiento del habla (*Speech Processing*)
- Síntesis de voz (*Speech Synthesis*)
- Razonamiento matemático
- Recuperación de información (*Information Retrieval*)
- Reconocimiento de entidades nombradas (*Named Entity Recognition*, NER) — por ejemplo, identificar en un texto entidades como personas, organizaciones, lugares o fechas.

1.4 Grandes Modelos de Lenguaje (LLMs)

Los **Large Language Models (LLMs)** son redes neuronales profundas de gran escala especializadas en el procesamiento del lenguaje. Existe todo un "árbol evolutivo" de modelos (tanto de código abierto como cerrado) que ha ido creciendo

desde 2018 hasta la actualidad, con familias como BERT, GPT, LLaMA, Claude, PaLM, entre muchas otras.

1.5 “Todo es álgebra”

Una forma humorística (pero certera) de resumir el *machine learning*: se introducen los datos en un gran conjunto de álgebra lineal, se “agita” (entrena) hasta que las respuestas empiezan a parecer correctas, y se recogen los resultados al otro lado. En el fondo, una red neuronal no es más que una composición de operaciones algebraicas (productos matriciales, sumas y funciones no lineales).

1.1 El Perceptrón

Unidad mínima de cómputo de una NN: neurona o perceptrón

El **perceptrón** (o neurona artificial) es la unidad de cómputo más pequeña de una red neuronal. Recibe un vector de entrada y produce una salida:

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^n \quad \longrightarrow \quad y \in \mathbb{R}$$

Componentes: vector de pesos y sesgo

El perceptrón consiste en un **vector de pesos** ($\vec{w}$) y un **sesgo** o **bias** (b):

$$\vec{w} \in \mathbb{R}^n \qquad b \in \mathbb{R}$$

La salida del perceptrón, antes de aplicar la función de activación, es el producto escalar entre la entrada y los pesos, más el sesgo:

$$y = \vec{x} \cdot \vec{w} + b$$

Función de activación

La función de activación **introduce el fenómeno de la no linealidad**. Sin ella, la composición de varias capas seguiría siendo equivalente a una única transformación lineal, lo cual limitaría enormemente la capacidad expresiva de la red.

$$y = f(\vec{x} \cdot \vec{w} + b)$$

donde f es la función de activación.

Tabla de funciones de activación habituales

Función de activación	Ecuación	Ejemplo de uso
Escalón unitario (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0 \\ 0.5, & z = 0 \\ 1, & z > 0 \end{cases}$	Variante de perceptrón
Signo (*Signum*)	$\phi(z) = \begin{cases} -1, & z < 0 \\ 0, & z = 0 \\ 1, & z > 0 \end{cases}$	Variante de perceptrón
Lineal	$\phi(z) = z$	Adaline, regresión lineal

Función de activación	Ecuación	Ejemplo de uso
Lineal a tramos (*Piece-wise linear*)	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2} \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2} \\ 0, & z \leq -\frac{1}{2} \end{cases}$	Máquinas de vectores de soporte (SVM)
Logística (sigmoide)	$\phi(z) = \dfrac{1}{1 + e^{-z}}$	Regresión logística, NN multicapa
Tangente hiperbólica	$\phi(z) = \dfrac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Redes neuronales multicapa
Rectificador, ReLU (*Rectified Linear Unit*)	$\phi(z) = \max(0, z)$	Redes neuronales multicapa
Rectificador, Softplus	$\phi(z) = \ln(1 + e^{z})$	Redes neuronales multicapa

1.2 Redes Neuronales

Unión de capas de neuronas

Una **red neuronal** es la unión de varias **capas** (*layers*) de neuronas conectadas entre sí. Cada capa transforma su entrada en una salida que sirve de entrada a la siguiente capa:

$$\vec{x} \in \mathbb{R}^n \longrightarrow \text{ capas ocultas } \left(\mathbb{R}^{n_1}, \mathbb{R}^{n_2}, \dots, \mathbb{R}^{n_k}\right) \longrightarrow \vec{y} \in \mathbb{R}^m$$

Un ejemplo clásico de uso es el reconocimiento de dígitos manuscritos (dataset MNIST): a partir de una imagen de un dígito, la red produce como salida el dígito reconocido (p. ej. “0”).

Aprendizaje profundo: capas

Una NN se considera **profunda** (*deep*) si tiene **3 o más capas**. Se distingue así entre:

- **NN**: red neuronal “simple”, con una capa de entrada conectada directamente a la de salida.
- **DNN** (*Deep Neural Network*): red con una o más capas intermedias entre la entrada y la salida.

Capas ocultas

Las capas que no son ni de entrada ni de salida se denominan **capas ocultas** (*hidden layers*).

La primera capa oculta se calcula como:

$$\vec{h}_1 = f(W_1\vec{x} + \vec{b}_1), \qquad \vec{h}_1 \in \mathbb{R}^{n_1}$$

De forma general, cada capa oculta k se calcula a partir de la salida de la capa anterior:

$$\vec{h}_k = f(W_k\vec{h}_{k-1} + \vec{b}_k), \qquad \vec{h}_k \in \mathbb{R}^{n_k}$$

Formalización completa de una red neuronal

Dado un vector de entrada $\vec{x}$, el proceso completo (*forward pass*) de una red con k capas es:

$$
\begin{aligned}
\vec{x} &= \vec{h}_0 && \text{(Entrada)} \\
\vec{h}_1 &= f(W_1\vec{h}_0 + \vec{b}_1) && \text{(Primera capa)} \\
\vec{h}_2 &= f(W_2\vec{h}_1 + \vec{b}_2) && \text{(Segunda capa)} \\
&\vdots \\
\vec{h}_k &= f(W_k\vec{h}_{k-1} + \vec{b}_k) && \text{(Capa final, salida)} \\
\vec{y} &= \vec{h}_k
\end{aligned}
$$

Donde, para cada capa i:

- W_i es la **matriz de pesos** de la capa i
- $\vec{b}_i$ es el **vector de sesgos** (*bias*) de la capa i
- f es la **función de activación** aplicada elemento a elemento

1.3 Tipos de salidas

Tipos de problemas

A medida que avanza la tecnología, el número de problemas que se pueden resolver con redes neuronales aumenta, por lo que enumerar una lista completa resulta inútil. Sin embargo, se pueden identificar **dos grandes tipos de problemas**:

- Problemas de **regresión**
- Problemas de **clasificación**

Problemas de regresión

En los problemas de regresión, la salida de la red neuronal es **un número real**, o un vector de números reales. Ejemplos: temperatura, divisa/moneda, tiempo, masa, número de personas, etc.

Se utilizan redes neuronales cuando los ejemplos de entrada contemplan múltiples variables, como datos de clientes, condiciones del terreno, previsiones meteorológicas, etc.

La **dimensión de entrada** corresponde al número de variables de los ejemplos de entrada, y la **dimensión de salida** es igual al número de valores reales que se desea predecir.

Problemas de clasificación

En los problemas de clasificación, en cambio, la salida de la red debe predecir **la categoría** de una entrada dada.

La salida de la NN en un problema de clasificación debe representar **la probabilidad** de que la entrada dada pertenezca a una clase específica.

Por tanto, la **dimensión de salida** debe ser igual al **número de clases** a identificar.

Salida de la red

La salida de una red neuronal es una función **no lineal** que depende de muchas variables internas (W y b):

$$\vec{y} = \mathbf{F}(\vec{x})$$

Sin embargo, esta salida **no es, por sí misma, una distribución de probabilidad**. Por ejemplo:

$$\vec{y} = (1.5 \quad 0.2 \quad -1.1 \quad 2.3 \quad -0.7 \quad ...)$$

Una distribución de probabilidad debe cumplir:

$$0 \leq p_i \leq 1 \qquad \sum_i p_i = 1$$

La función Softmax

Para obtener una distribución de probabilidad a partir de la salida de la red (los llamados ***logits***), se utiliza habitualmente la función **Softmax**:

$$p_i = \frac{e^{y_i}}{\sum_{j=1}^{m} e^{y_j}}$$

Por ejemplo, dado el vector de *logits*:

$$\vec{y} = \begin{pmatrix} 1.5 \\ 0.2 \\ -1.1 \\ 2.3 \\ -0.7 \\ \vdots \end{pmatrix}$$

Tras aplicar Softmax se obtiene una distribución de probabilidad válida:

$$\vec{p} = \begin{pmatrix} 0.25 \\ 0.11 \\ 0.02 \\ 0.51 \\ 0.06 \\ \vdots \end{pmatrix}$$

En este ejemplo, la clase con mayor probabilidad (0.51) sería la predicción de la red.

Función de activación según el tipo de problema

Clasificación

En el caso de un problema de **clasificación** (multiclase, con clases mutuamente excluyentes), la función de activación de la capa de salida debe ser una **Softmax**, para obtener una distribución de probabilidad.

Sin embargo, otros tipos de problemas requieren funciones de activación distintas.

Regresión

Para un problema de **regresión**, no es necesario aplicar ninguna función de activación en la salida, ya que la red debe poder producir directamente números reales (positivos o negativos).

En el caso de una **regresión estrictamente positiva** (es decir, cuando la salida debe ser siempre ≥ 0), se pueden usar funciones como:

- **ReLU**: $\max(x, 0)$
- **Leaky ReLU**: $\max(x, ax)$, con $0 < a < 1$
- **Parametric ReLU (PReLU)**: variante de Leaky ReLU donde el parámetro a se aprende durante el entrenamiento
- **Softplus**: $\ln(1 + e^x)$

Comparativa:

- **ReLU** suele ser más rápida, pero puede producir gradientes nulos (0) para las salidas negativas (el problema conocido como "neuronas muertas").
- **Leaky ReLU** soluciona el problema de los gradientes nulos, pero requiere fijar un parámetro (o entrenarlo mediante PReLU).
- **Softplus** es computacionalmente más costosa que las anteriores.

Clasificación binaria

Para un problema de **clasificación binaria**, la salida puede ser 0/1. En estos casos se puede usar una función **sigmoide**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Esta función comprime (*squash*) las salidas de la red al rango $(0, 1)$, dando una probabilidad más alta cuanto más se aleja el valor de 0, y exactamente 0.5 quando el valor de entrada es 0.

Clasificación multietiqueta (*multilabel*)

Para un problema de **clasificación multietiqueta**, en el que la salida puede pertenecer a varias etiquetas simultáneamente, el problema es equivalente a un problema de clasificación binaria con múltiples salidas independientes. Por ello, también se puede utilizar una función **sigmoide** (aplicada de forma independiente a cada salida, en lugar de Softmax, ya que las clases no son mutuamente excluyentes).

Función de activación en las capas ocultas

Para las **capas ocultas**, la función de activación estándar es **ReLU**, o alguna variante ligeramente modificada como **Leaky ReLU** o **PReLU**.

Resumen final

Tipo de problema	Función de activación en la salida
Regresión (valores reales, con signo)	Ninguna (activación lineal / identidad)
Regresión estrictamente positiva	ReLU, Leaky ReLU, PReLU, Softplus
Clasificación binaria	Sigmoide
Clasificación multiclase (excluyente)	Softmax
Clasificación multietiqueta	Sigmoide (por salida)
Capas ocultas (en general)	ReLU / Leaky ReLU / PReLU

Apuntes elaborados a partir de la presentación "Introduction to Neural Networks — Data Mining III" de Ander González Docasal, Mondragon Unibertsitatea (Escuela Politécnica Superior).